

Experiment 'p_4_2_c_cumulative_totals_array' Results

June 5, 2026

Experiment outcome: SUCCESS

Bad responses: 0

Responses containing assume : 0

Responses containing expect : 0

Resolution attempts: 3

Hard fails (resolution): 0

Soft fails (resolution): 0

Verification attempts: 3

Problem Specification

Problem name: p_4_2_c_cumulative_totals_array

Natural language statement: null

Method signature: p_4_2_c_cumulative_totals_array(inputs: array<int>) returns (totals: array<int>)

Ensures

- totals.Length == inputs.Length
- forall i :: 0 <= i < inputs.Length ==> totals[i] == (if i == 0 then inputs[0] else totals[i - 1] + inputs[i])

Requires

- 0 < inputs.Length

GenAI interactions

Below you will find all interactions between the ‘user’ (program) and the ‘assistant’ (OpenAI).

Program → GenAI

You are given the following task to perform in Dafny:

Write a method to satisfy the below requirements.

The signature should be:

`p_4_2_c_cumulative_totals_array(inputs: array<int>) returns (totals: array<int>)`

The method should respect the following contract:

requires $0 < \text{inputs.Length}$, ensures $\text{totals.Length} == \text{inputs.Length}$, ensures forall $i :: 0 \leq i < \text{inputs.Length} \Rightarrow \text{totals}[i] == (\text{if } i == 0 \text{ then } \text{inputs}[0] \text{ else } \text{totals}[i - 1] + \text{inputs}[i])$

Produce and show only the Dafny body of this method, including the curly braces that surround it. Do not show the signature nor contract. You must not use 'assume' nor 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-DnLNBDqeH8HJyGG1tMvD9tay45fRM

Model: gpt-5.5-2026-04-23

Created at: 1780652601

Finish reason: stop

Usage: 348 tokens in, and 640 tokens out

```
{
  totals := new int[inputs.Length];
  totals[0] := inputs[0];

  var i := 1;
  while i < inputs.Length
    invariant 1 <= i <= inputs.Length
    invariant totals.Length == inputs.Length
    invariant forall j :: 0 <= j < i ==> totals[j] == (if j == 0 then inputs[0] else
      ↪ totals[j - 1] + inputs[j])
  {
    totals[i] := totals[i - 1] + inputs[i];
    i := i + 1;
  }
}
```

Program → GenAI

When using dafny verify, the below error is emitted and verify fails:

```
problem_data/p_4_2_c_cumulative_totals_array/program.dfy(13,14): Error: this invariant
  ↪ could not be proved to be maintained by the loop
Related message: loop invariant violation
|
13 | invariant forall j :: 0 <= j < i ==> totals[j] == (if j == 0 then inputs[0] else
  ↪ totals[j - 1] + inputs[j])
  | ~~~~~
```

Dafny program verifier finished with 1 verified, 1 error

Correct the error in code by altering only the method body. Produce and show only the
 ↪ Dafny body, including the curly braces that surround it. Do not show the signature
 ↪ nor contract. You must not use 'assume' nor 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-DnLNbYzcdtpTyKgJCl27MsGjYUraQ

Model: gpt-5.5-2026-04-23

Created at: 1780652627

Finish reason: stop

Usage: 694 tokens in, and 3635 tokens out

```
{
  totals := new int[inputs.Length];
  totals[0] := inputs[0];

  var i := 1;
  while i < inputs.Length
    invariant 1 <= i <= inputs.Length
    invariant totals.Length == inputs.Length
    invariant forall j :: 0 <= j < i ==> totals[j] == (if j == 0 then inputs[0] else
      ↪ totals[j - 1] + inputs[j])
  {
    var oldI := i;
    ghost var prev := totals[..oldI];

    assert forall j :: 0 <= j < oldI ==> prev[j] == (if j == 0 then inputs[0] else prev[j
      ↪ - 1] + inputs[j]) by {
      forall j | 0 <= j < oldI
        ensures prev[j] == (if j == 0 then inputs[0] else prev[j - 1] + inputs[j])
      {
        assert prev[j] == totals[j];
        if j == 0 {
          assert totals[j] == inputs[0];
        } else {
          assert 0 <= j - 1 < oldI;
          assert prev[j - 1] == totals[j - 1];
        }
      }
    }
  }
}
```

```

        assert totals[j] == totals[j - 1] + inputs[j];
    }
}

totals[oldI] := totals[oldI - 1] + inputs[oldI];
assert totals[..oldI] == prev;

i := oldI + 1;

assert forall j :: 0 <= j < i ==> totals[j] == (if j == 0 then inputs[0] else totals[j
↪ - 1] + inputs[j]) by {
  forall j | 0 <= j < i
    ensures totals[j] == (if j == 0 then inputs[0] else totals[j - 1] + inputs[j])
  {
    if j == oldI {
      assert j != 0;
      assert totals[j] == totals[j - 1] + inputs[j];
    } else {
      assert j < oldI;
      assert totals[j] == prev[j];
      if j == 0 {
        assert prev[j] == inputs[0];
      } else {
        assert 0 <= j - 1 < oldI;
        assert totals[j - 1] == prev[j - 1];
        assert prev[j] == prev[j - 1] + inputs[j];
      }
    }
  }
}
}
}
}
}

```

Program → GenAI

When using dafny verify, the below error is emitted and verify fails:

```

problem_data/p_4_2_c_cumulative_totals_array/program.dfy(24,10): Error: assertion might
↪ not hold

```

```

|
24 | assert totals[j] == inputs[0];
| ~~~~~

```

```

problem_data/p_4_2_c_cumulative_totals_array/program.dfy(49,12): Error: assertion might
↪ not hold

```

```

|
49 | assert prev[j] == inputs[0];
| ~~~~~

```

Dafny program verifier finished with 1 verified, 2 errors

Correct the error in code by altering only the method body. Produce and show only the
↪ Dafny body, including the curly braces that surround it. Do not show the signature
↪ nor contract. You must not use 'assume' nor 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-DnLOpEtpB931e6Of4XDi18B7N0K8Y

Model: gpt-5.5-2026-04-23

Created at: 1780652703

Finish reason: stop

Usage: 1482 tokens in, and 2543 tokens out

```
{
  totals := new int[inputs.Length];
  totals[0] := inputs[0];

  var i := 1;
  while i < inputs.Length
    invariant 1 <= i <= inputs.Length
    invariant totals.Length == inputs.Length
    invariant totals[0] == inputs[0]
    invariant forall j :: 1 <= j < i ==> totals[j] == totals[j - 1] + inputs[j]
  {
    var n := i;
    totals[n] := totals[n - 1] + inputs[n];

    assert n != 0;
    assert totals[0] == inputs[0];

    assert forall j :: 1 <= j < n + 1 ==> totals[j] == totals[j - 1] + inputs[j] by {
      forall j | 1 <= j < n + 1
        ensures totals[j] == totals[j - 1] + inputs[j]
      {
        if j == n {
          assert totals[j] == totals[j - 1] + inputs[j];
        } else {
          assert j < n;
          assert 1 <= j < i;
          assert totals[j] == totals[j - 1] + inputs[j];
        }
      }
    }

    i := n + 1;
  }

  assert i == inputs.Length;
  assert forall k :: 0 <= k < inputs.Length ==> totals[k] == (if k == 0 then inputs[0]
    ↪ else totals[k - 1] + inputs[k]) by {
```

```

forall k | 0 <= k < inputs.Length
  ensures totals[k] == (if k == 0 then inputs[0] else totals[k - 1] + inputs[k])
{
  if k == 0 {
    assert totals[k] == inputs[0];
  } else {
    assert 1 <= k < i;
    assert totals[k] == totals[k - 1] + inputs[k];
  }
}
}
}

```

Final Program

```

method p_4_2_c_cumulative_totals_array(inputs: array<int>) returns (totals: array<int>)
  requires 0 < inputs.Length
  ensures totals.Length == inputs.Length
  ensures forall i :: 0 <= i < inputs.Length ==> totals[i] == (if i == 0 then inputs
    ↪ [0] else totals[i - 1] + inputs[i])
{
  totals := new int[inputs.Length];
  totals[0] := inputs[0];

  var i := 1;
  while i < inputs.Length
    invariant 1 <= i <= inputs.Length
    invariant totals.Length == inputs.Length
    invariant totals[0] == inputs[0]
    invariant forall j :: 1 <= j < i ==> totals[j] == totals[j - 1] + inputs[j]
  {
    var n := i;
    totals[n] := totals[n - 1] + inputs[n];

    assert n != 0;
    assert totals[0] == inputs[0];

    assert forall j :: 1 <= j < n + 1 ==> totals[j] == totals[j - 1] + inputs[j] by {
      forall j | 1 <= j < n + 1
        ensures totals[j] == totals[j - 1] + inputs[j]
      {
        if j == n {
          assert totals[j] == totals[j - 1] + inputs[j];
        } else {
          assert j < n;
          assert 1 <= j < i;
          assert totals[j] == totals[j - 1] + inputs[j];
        }
      }
    }
  }
}

```

```

    i := n + 1;
  }

  assert i == inputs.Length;
  assert forall k :: 0 <= k < inputs.Length ==> totals[k] == (if k == 0 then inputs[0]
    ↪ else totals[k - 1] + inputs[k]) by {
    forall k | 0 <= k < inputs.Length
      ensures totals[k] == (if k == 0 then inputs[0] else totals[k - 1] + inputs[k])
    {
      if k == 0 {
        assert totals[k] == inputs[0];
      } else {
        assert 1 <= k < i;
        assert totals[k] == totals[k - 1] + inputs[k];
      }
    }
  }
}

```

Total Token Usage

Input tokens: 2524

Output tokens: 6818

Reasoning tokens: 5544

Sum of ‘total tokens’: 9342

Experiment Timings

Overall Experiment started at 1780652600974, ended at 1780652769159, lasting 168185ms (168.19 seconds)

Iteration #1 started at 1780652600975, ended at 1780652626693, lasting 25718ms (25.72 seconds)

Iteration #2 started at 1780652626693, ended at 1780652703361, lasting 76668ms (76.67 seconds)

Iteration #3 started at 1780652703362, ended at 1780652769159, lasting 65797ms (65.80 seconds)